

Hi,

After cleaning our data in the previous module, here we're going to use the prepared data to generate first business insight.

Exercise 1 - events detection

Your marketing department prepared a special offer for all the visitors of your website using the macOS. Write a streaming pipeline that will:

- read data continuously from *visits* topic
 - the schema contains the following fields:
 - eventId: string
 - os: string, possible values: "macOS", "Windows", "Linux"
 - visitedPage: string
- filter out all visitors not using the "macOS" system
- add an attribute *processedAt* with the current processing time to the "macOS" events
- write the result as JSON to `/tmp/bde/module5/homework/e1/output`
 - the result should be a JSON structure composed of the following fields:
 - eventId, os, visitedPage, processedAt, for example:

```
{"eventId":"505306e5-b41d-466b-b839-f56903a1afa6","os":"macOS","visitedPage":"category-1/product-1","processedAt":"2023-06-01T03:52:23.949+00:00"}
```

The Kafka connection parameters for Apache Spark:

- kafka.bootstrap.servers: localhost:29092
- subscribe: visits
- startingOffsets: earliest

Setup instructions

Java/Scala

1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/scala> in IntelliJ.

2. Implement your job.

3. Go to `cd docker/p1` and start the containers (docker-compose down --volumes; docker-compose up).

4. Go to `com.becomedataengineer.exercise1.DataGenerator` and run it.

5. The DataGenerator should continuously write random visits to the *visits* topic and your job should write MacOS users to JSON files. You can check the content of the files with this command `tail -f /tmp/bde/module5/homework/e1/output\/*`

Python

1. Open

<https://github.com/bartos25/master-stream-processing-homework/tree/master/03-stream-processing/python> in PyCharm.

2. Implement your job.

3. Go to `cd docker/p1` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Go to `exercise1/data_generator.py` and run it.

5. The `data_generator` should continuously write random visits to the *visits* topic and your job should write MacOS users to JSON files. You can check the content of the files with this command `tail -f /tmp/bde/module5/homework/e1/output/*`

Exercise 2 - data reprocessing

Your data cleansing job writes the processing outcome to an Apache Kafka topic and a directory on the file system. The produced data is later consumed by other streaming and batch jobs. One of these streaming jobs is yours.

In the most recent release you had deployed a buggy version of the code but saw that only 3 days later. As a result, you must reprocess the last 3 days.

Write a data reprocessing pipeline that will:

- read the data from your local file system
 - data is stored under `/tmp/bde/module5/homework/e2/output`
 - partitions are defined as `event_time=${year-month-day}`, for example `event_time=2023-06-03`, e.g. to reprocess the data for 2023-06-03, you need to read `/tmp/bde/module5/homework/e2/output/event_time=2023-06-03`
 - data is written as stringified JSON, e.g.
`{"value":{"eventId":"eba744ef-90ae-4c6e-8367-11c1e8a3c77a","os":"Linux","visitedPage":"index.html"}}`
- process the batch data with the same processing logic as the streaming job present in the `StreamingJob.scala` or `streaming_job.py` file
- write the result to the same location as the real-time streaming job

The goal of the exercise is to maximize the code sharing.

The Kafka connection parameters for Apache Spark:

- input data:
 - `kafka.bootstrap.servers: localhost:29092`

- subscribe: visits_raw
 - startingOffsets: earliest
- output data:
 - kafka.bootstrap.servers: localhost:29092
 - topic: valid_visits

Setup instructions

Java/Scala

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/scala> in IntelliJ.
2. Refactor the real-time job and implement the reprocessing job.
3. Go to cd docker/p2 and start the containers (docker-compose down --volumes; docker-compose up).
4. Generate some data by running com.becomedataengineer.exercise2.DataGenerator.
5. The DataGenerator should write rows for the last 10 days to visits_raw topic and /tmp/bde/module5/homework/e2/output directory. Wait for it to finish (should print "**Finished!**" message).
6. Start a console consumer for the target topic: docker exec -ti p2_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic valid_visits --from-beginning
7. Run your real-time job. You should see the records in the console consumer.
8. Choose a dates range and run the reprocessing job. It should also write the records to the valid_visits topic. To define the time range for reprocessing, list all available partitions with ls /tmp/bde/module5/homework/e2/output/

Python

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/python> in PyCharm.
2. Implement your job.
3. Go to cd docker/p2 and start the containers (docker-compose down --volumes; docker-compose up).
4. Go to exercise2/data_generator.py and run it.
5. The data_generator should write rows for the last 10 days to visits_raw topic and /tmp/bde/module5/homework/e2/output directory. Wait for it to finish (should print "**Finished!**" message).
6. Start a console consumer for the target topic: docker exec -ti p2_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic valid_visits --from-beginning
7. Run your real-time job. You should see the records in the console consumer.

8. Choose a dates range and run the reprocessing job. It should also write the records to the `valid_visits` topic. To define the time range for reprocessing, list all available partitions with `ls /tmp/bde/module5/homework/e2/output/`

Exercise 3 - Dead-Letter Queue

From time to time one of your streaming jobs receives data that cannot be successfully processed. Whenever the mapping function fails converting the incoming records, there is a fatal failure and the job stops. Since we're talking here about a long-running streaming job, any failure is not acceptable.

To solve the issue implement the Dead-Letter Queue within the job that will:

- modify the mapping so that an exception doesn't cause the fatal failure
- write the failing raw data to the `dead_letter` topic and the successfully processed data to the `valid_data` topic

The job is located in `com.becomedataengineer.exercise3.DeadLetterQueueJob` (Scala) and `com.becomedataengineer.exercise3.DeadLetterQueueJob`(Python) **TODO!!!**

Setup instructions

Java/Scala

1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/scala> in IntelliJ.

2. Go to `cd docker/p3` and start the containers (`docker-compose down --volumes;` `docker-compose up`).

3. Start the `com.becomedataengineer.exercise3.DataGenerator`.

4. Start the `com.becomedataengineer.exercise3.DeadLetterQueueJob`. It should critically fail after some runs with the exception like:

- ```
Caused by: org.apache.spark.SparkException: Job aborted due to stage failure:
Task 1 in stage 0.0 failed 1 times, most recent failure: Lost task 1.0 in stage
0.0 (TID 1) (192.168.1.55 executor driver): java.util.NoSuchElementException:
None.get
 at scala.None$.get(Option.scala:529)
 at scala.None$.get(Option.scala:527)
```

5. Stop `DataGenerator` and `DeadLetterQueueJob`.

6. Remove the checkpoint `rm -rf /tmp/bde/module5/homework/e3/checkpoints`

7. Go to `DeadLetterQueueJob` and adapt the map function to the dead-lettering requirement.

8. Open console consume for the `dead_letter` topic: `docker exec -ti p3_kafka`

`kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic dead_letter --from-beginning`

9. Open console consumer for the `valid_data` topic: `docker exec -ti p3_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic valid_data --from-beginning`
10. Start the refactored `DeadLetterQueueJob`. It shouldn't be failing anymore and you should be seeing the records coming to the 2 topics (`valid_data`, `dead_letter`).

## Python

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/python> in PyCharm.
2. Go to `cd docker/p3` and start the containers (`docker-compose down --volumes;` `docker-compose up`).
3. Start the `exercise3/data_generator.py`
4. Start the `exercise3/dead_letter_queue_job.py`. It should critically fail after some runs with the exception like:

```
pyspark.errors.exceptions.captured.StreamingQueryException: [STREAM_FAILED]
Query [id = 295a9496-189e-4a48-a716-08a0a22d8973, runId =
0ce63a62-00a5-4649-9e0b-d169c832ce2d] terminated with exception: Job aborted
due to stage failure: Task 1 in stage 0.0 failed 1 times, most recent failure:
Lost task 1.0 in stage 0.0 (TID 1) (192.168.1.55 executor driver):
org.apache.spark.api.python.PythonException: Traceback (most recent call last):
 File "/home/bartosz/workspace/bde-homework-5/python/exercise3/mappers.py",
 line 15, in get_visited_page
 values_to_return[index] = json.dumps({'page': json_dict["visitedPage"]})
 KeyError: 'visitedPage'

 at
org.apache.spark.api.python.BasePythonRunner$ReaderIterator.handlePythonException
on(PythonRunner.scala:561)
 at
org.apache.spark.sql.execution.python.PythonArrowOutput$$anon$1.read(PythonArrow
wOutput.scala:118)
 at
org.apache.spark.api.python.BasePythonRunner$ReaderIterator.hasNext(PythonRunne
r.scala:514)
 at
org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator.scala:37)
```

5. Stop `data_generator.py` and `dead_letter_queue_job.py`.
6. Remove the checkpoint `rm -rf /tmp/bde/module5/homework/e3/checkpoints`
7. Go to `dead_letter_queue_job.py` and adapt the `get_visited_page` function to the dead-lettering requirement.
8. Open console consume for the `dead_letter` topic: `docker exec -ti p3_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic dead_letter --from-beginning`
9. Open console consumer for the `valid_data` topic: `docker exec -ti p3_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic valid_data --from-beginning`
10. Start the `dead_letter_queue_job.py`. It shouldn't be failing anymore and you should be seeing the records coming to the 2 topics (`valid_data`, `dead_letter`).

## Exercise 4 - backpressure

Your Apache Spark Structured Streaming job works in micro-batch mode. Unfortunately, you see more and more performance issues due to the delivery latency of the massively arriving late data.

Implement a backpressure mechanism that for each micro-batch will:

- combine the backpressure data with the real-time data
  - the records are ordered by the *generation\_time* field; use it in the backpressure logic
- print only the first 5 records to the console output sink
- save the remaining ones to files on your local file system for the next micro-batch

The Kafka connection parameters for Apache Spark:

- kafka.bootstrap.servers: localhost:29092
- subscribe: data\_for\_backpressure
- startingOffsets: earliest

The record schema is:

- id: int
- row\_value: string
- generation\_time: timestamp

### Setup instructions

Java/Scala

1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/scala> in IntelliJ.

2. Implement your job with the backpressure mechanism.

3. Go to `cd docker/p4` and start the containers (`docker-compose down --volumes;`  
`docker-compose up`).

4. Go to `com.becomedataengineer.exercise4.DataGenerator` and run it.

5. The `DataGenerator` should continuously write 7 records to the `data_for_backpressure` topic.

6. Test your job. Your micro-batch should print 5 records, each time for 2 different minutes:

```
+-----+
|id|row_value|generation_time|
+-----+
1	EE	2023-06-24 04:00:48.69
1	A	2023-06-24 04:01:43.69
1	E	2023-06-24 04:01:44.69
1	I	2023-06-24 04:01:45.69
1	U	2023-06-24 04:01:46.69
+-----+
```

## Python

### 1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/03-stream-processing/python> in PyCharm.

2. Implement your job with the backpressure mechanism.

3. Go to `cd docker/p4` and start the containers (`docker-compose down --volumes;`  
`docker-compose up`).

4. Go to `exercise4/data_generator.py` and run it.

5. The `data_generator.py` should continuously write 7 records to the `data_for_backpressure` topic.

6. Test your job. Your micro-batch should print 5 records, each time for 2 different minutes:

```
+-----+
|id|row_value|generation_time|
+-----+
1	EE	2023-06-24 04:00:48.69
1	A	2023-06-24 04:01:43.69
1	E	2023-06-24 04:01:44.69
1	I	2023-06-24 04:01:45.69
1	U	2023-06-24 04:01:46.69
+-----+
```

 **If you are stuck**, please check the *Homework - instructions* document present in the same page. It contains some extra details on how to implement various parts of the homework.

Have fun and see you in the next module!

Bartosz.